

INCIDENTE · Login muestra 'suscripción no activa' a usuarios con plan pago

Owner: Celestino · 2026-05-02 · Severidad **P0/P1** · Branch **hotfix/auth-multi-customer-match** · Base **main@84466d9** · Modo: **read-only diagnóstico + hotfix en rama, sin merge**

Estado: Causa raíz identificada en el código (sin necesidad de Stripe API porque el bug es estático). Hotfix implementado en rama, **12/12 tests passing**, tsc/build/lint verdes. **NO mergeado · NO desplegado · CERO writes** a Stripe/DB/env/WhatsApp/email.

1. Causa raíz · DOS bugs combinados en landing/auth.ts

Bug A · `customers.list({ email, limit: 1 })`

Stripe permite tener **varios customers con el mismo email**. Esto pasa naturalmente cuando un usuario hace checkout sin estar logueado, lo que ocurre cada vez que vuelve al landing y paga de nuevo (Customer Portal de Stripe a veces crea uno nuevo si el flow lo permite). El código previo (*auth.ts:24*):

```
const customers = await getStripe().customers.list({
  email,
  limit: 1,
});
if (customers.data.length === 0) return null;
const customer = customers.data[0];
```

Solo mira el primer customer que Stripe devuelve. Si la suscripción activa quedó en otro customer (más nuevo), el login rechaza al usuario aunque sí tenga plan pago.

Bug B · `subscriptions.list` filtrando solo `status='active'`

```
const subscriptions = await getStripe().subscriptions.list({
  customer: customer.id,
  status: 'active',
  limit: 1,
});
if (subscriptions.data.length === 0) return null;
```

Trialing (trial vigente) y **past_due** (pago falló pero sub aún no cancelada) quedan fuera. **Past_due** es particularmente peligroso porque es el estado donde el Customer Portal sirve para que el usuario actualice su tarjeta — al cerrarles el dashboard, lo único que pueden hacer es escribir al soporte.

Bug C · UX del mensaje de error

app/login/page.tsx traducía CUALQUIER fallo del *authorize* (sin customer · password incorrecto · sub no encontrada · email vacío) al mismo mensaje:

```
Tu suscripción no está activa.
Selecciona un plan para continuar.
[Ver planes disponibles] → /#pricing
```

Empuja a pagar de nuevo a usuarios que YA pagaron. Doble cobro accidental como riesgo secundario.

2. ¿El password se valida?

Sí. En *auth.ts:38-41* previo al hotfix:

```
const expected = deriveDashboardPassword(customer.id);
if (!expected || !constantTimeEqual(password, expected)) {
  return null;
}
```

Validación correcta y en tiempo constante (T1.4.B). El password se deriva con HMAC-SHA256(customer_id, DASHBOARD_PASSWORD_SECRET) → primer 12 chars hex con prefijo *dona-*. **El password NO se ignora.** Pero la validación ocurría sobre el customer EQUIVOCADO (el primero que Stripe devolvió).

3. Diagnóstico Stripe API

No ejecutado: *STRIPE_SECRET_KEY* no está disponible en el entorno local del verificador (las claves viven en Vercel/Render, no en el repo). Esto es lo correcto · cumple la instrucción del incidente: no revelar tokens.

Si quieres confirmar la cantidad real de customers para *celestinojbm@gmail.com*, ejecutar tú localmente con *STRIPE_SECRET_KEY* del dashboard de Stripe:

```
# Listar todos los customers con ese email (read-only)
stripe customers list --email celestinojbm@gmail.com --limit 100

# Para cada cus_XXX devuelto, listar TODAS sus subs
stripe subscriptions list --customer cus_XXX --status all
```

Esperado según la hipótesis: ≥ 2 customers, donde la sub *active* está en uno distinto al que *customers.list({email,limit:1})* devolvía primero. **Importante:** solo el owner debe correr esto · no compartir output con tokens/IDs completos.

4. Hotfix implementado en la rama

Archivo	Cambio
landing/lib/auth-matcher.ts	NUEVO · función pura <i>encontrarCustomerConSub</i> con dependencias inyectables. It
landing/lib/auth-matcher.test.ts	NUEVO · 12 tests con stubs de Stripe (sin tocar API real).
landing/auth.ts	Reemplazado el authorize por una delegación al matcher. Side-effect positivo: limpiados los <i>(use
landing/app/login/page.tsx	Mensaje neutral · CTA cambia de '#pricing' a 'mailto:hola@usadona.com' (no empuja a pagar de nu
landing/package.json	+ scripts <i>test</i> y <i>test:watch</i> · + devDep <i>vitest@^2.1.9</i>.
landing/vitest.config.ts	NUEVO · resolve alias '@' para imports tipo '@lib/...' · include lib/**/*.test.ts.

Diff conceptual del authorize

```
# Antes
customers.list({ email, limit: 1 })
  → customer = data[0]
validate password against customer
subscriptions.list({ customer, status: 'active' })
  → sub = data[0]

# Después (hotfix)
customers.list({ email, limit: 100 })
for each customer in data:
  if deleted or email mismatch: skip
  expected = derivePassword(customer.id)
  if password != expected: continue
  subs = subscriptions.list({ customer, status: 'all' })
  sub = first sub with status in {active, trialing, past_due}
  if sub: return { customer, subscription: sub }
return null
```

5. Tests ejecutados

npm test · 12/12 passing · 336ms · vitest@2.1.9

#	Test	Cubre
1	email sin customers → null	Sanity baseline
2	1 customer con sub canceled → null	Política · canceled NO autoriza
3	2 customers, sub activa en el segundo → match	Caso real del incidente
4	password incorrecto en cus_1, correcto en cus_2 → match	Interacción respeta password por customer
5	sub trialing → autorizada	Política · trialing OK
6	sub past_due → autorizada	Política · past_due OK · UX dashboard ofrece Customer Portal
7	sub incomplete → rechazada	Política · incomplete NO autoriza
8	customer deleted → saltado	No considera customers borrados
9	DASHBOARD_PASSWORD_SECRET ausente → null	Sin secret no hay path permisivo
10	email vacío → null	Validación de input
11	password vacío → null	Validación de input

6. Validaciones complementarias

Comando (cwd: landing/)	Resultado
<code>npx tsc --noEmit</code>	0 errores · exit 0
<code>rm -rf .next && npm run build</code>	OK · 17/17 pages · static + dynamic
<code>npm run lint</code>	3 warnings · 0 errors · exit 0 (antes: 6 errors + 3 warnings · exit 1) · los 6
<code>npm test</code>	12/12 passing · vitest@2.1.9

Sobre las dependencias agregadas

Solo **vitest@^2.1.9** como devDependency. *npm audit* reporta 6 moderate (esbuild/vite chain · solo dev server, no afecta bundle prod) y 1 high preexistente en Next.js 16.2.2 (no relacionado con el hotfix). **No se ejecutó *audit fix --force*** para no introducir cambios ortogonales.

7. Confirmaciones explícitas (sin writes)

NO se ejecutó nada de esto:

- Stripe writes (customers.update, subscriptions.cancel, refund, delete...)
- DB writes (Render Postgres / SQLite) · de hecho ni siquiera se conectó al backend Render
- Cambios de variables de entorno (Vercel / Render / .env local)
- Deploys manuales (Vercel / Render)
- Envíos de WhatsApp (Whapi / Meta / Twilio)
- Envíos de email
- *git push --force*, *git reset --hard*, ni merge a main

8. Riesgos del hotfix

Riesgo	Severidad	Mitigación
customers.list devuelve N>100 y el verdadero está en página 2	Baja	Stripe rara vez tiene 100+ customers con el mismo email · si pasa el ov
Iterar customers multiplica latencia del login · N llamadas a subscriptions.list	Baja	promedio esperado N=1-3. Worst case con 100 customers: ~100×0.2s
Aceptar past_due abre login a usuarios cuyo pago falló	Cero	es la política de Stripe · el dashboard tiene banner 'Pago atrasado' y CTA 'Gestionar facturación'
Side-effect: cambio del mensaje de error podría confundir a usuarios que no pagan	Baja	NO pagar · sigue diciendo 'No pudimos validar tu acceso' y 'si ya pagas
Vulnerabilidades npm audit nuevas	Cero	Vitest se usa solo en CI/local · esbuild/vite vulnerabilities afectan dev se
Refactor del any de callbacks rompe NextAuth	Cero	tsc 0 errores · build 17/17 OK · NextAuth strategy='jwt' sin cambios · tip

9. Próximos pasos · pendientes de tu aprobación

- **Tú:** revisar código de la rama *hotfix/auth-multi-customer-match* y ejecutar el comando *stripe* CLI de la sección 3 con tu STRIPE_SECRET_KEY **solo si quieres confirmar** que tu cuenta tiene 2+ customers (el fix es válido aunque no se confirme · el bug es estático).

- **Tú:** aprobar abrir PR contra main. La rama está pushed pero **no creé PR** (mismo issue de tokens vacíos que en T2.0.D · GH_TOKEN/GITHUB_TOKEN len=0).
- **Tú:** tras merge, redeploy automático Vercel y validar login con el usuario afectado en producción.
- **Tú:** si el hotfix funciona en prod, considerar PR follow-up para consolidar duplicados de customer en Stripe (tema de housekeeping, no funcional).

Hotfix listo para review. Cero merges, cero deploys. Tests verdes. Causa raíz documentada. UX mejorada.
Side-effect: lint exit 0 (antes 1).